

廈門大學

中 期 汇 报

OpenHarmony 向标联合查询中期汇报

汇报人：林正刚

年度课题-挑战方向3：向标联合查询性能优化



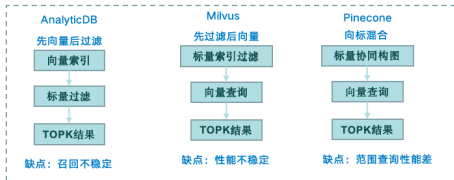
技术背景

端侧向量数据库（比如Ark Data自研数据库）通过标量向量数据的统一管理和联合查询，支撑：小艺记忆语义内容查询、小艺搜索本地数字资产语义问答。
为了在大数据集上快速查询，建立向量索引与标量索引，在联合查询场景下维护两套索引，若选择不好的执行计划，查询时延会大幅上升，对DB带来较大压力。

技术挑战

1. 端侧数据库考虑资源功耗问题没有统计信息，无法通过代价计算选择生成执行计划。
2. 构建轻量级混合索引，低底噪、低功耗、低时延查询。

当前结果



技术诉求

实现低功耗稳定低时延的向量标量联合查询。

1. 客观效率方面：百万128维向量与标量数据集下标量条件等值或者范围查询联合向量近似查询中，标量条件命中数量多少不影响时延召回率，10@10召回率>98%；
2. 主观感受方面：在对表中数据查询，对查询条件包含标量等值或这个范围查询并包含向量近似查询召回率与性能稳定，不会由于数据分布的问题导致部分查询出现劣化。

分解目标

动态增量混合索引：端侧数据是会不断增加更新，索引应可以从0数据量开始构建，增量更新数据不影响召回率。

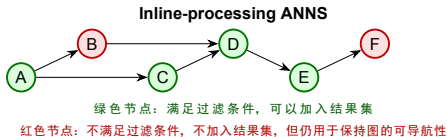
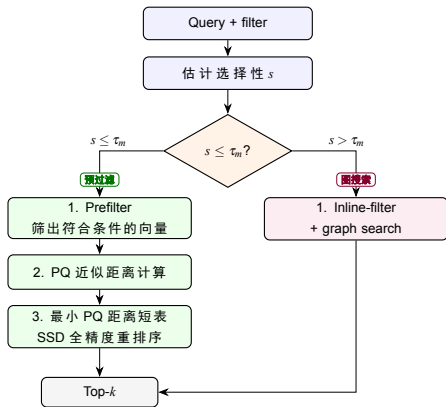
情景举例：在端侧用户创建表和索引构建后用户插入大量数据，查询数据满足标量条件以及最近似的查询向量top10，低延时稳定查出结果。

考量点：(1)数据不断增删改，查询10@10召回率稳定>98%；(2)索引能够支持标量等值与范围查询，联合向量近似查询时延<10ms；(3)页缓存内存限制10M左右；(4)索引膨胀率不能高于原始向量1倍。(5)基于mate60进行验证。

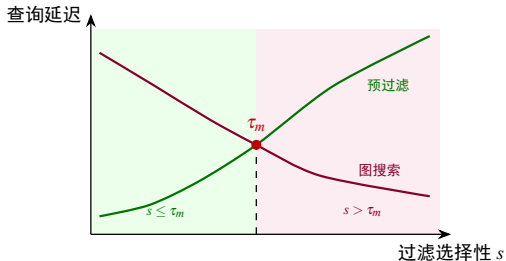
OpenAtom OpenHarmony

需求点	状态	备注
动态增量混合索引	完成	10k 以内走纯内存精确近邻搜索，超过阈值后落盘为磁盘图。
查询 10@10 召回率稳定 >98%	完成	
标量等值与范围查询	完成	
联合向量近似查询时延 <10ms	完成	
内存限制 30M 左右	完成	
索引膨胀率不能高于原始向量 1 倍	部分完成	R=116/PQ16 下 1M 额外索引约为原始向量 2.03 倍；主要来自磁盘图。

根据过滤选择性阈值 τ_m ，在同一个磁盘图索引上选择 prefilter 或 inline-processing ANNS。



标定曲线示意



曲线含义

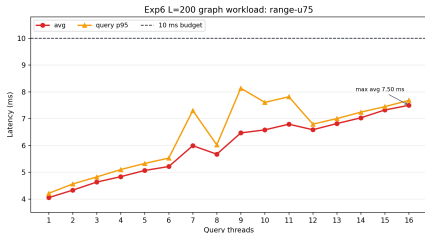
- 先过滤：选择性越高，候选越多，延迟上升。
- 图搜索：选择性越高，有效点越密，搜索范围缩小。
- τ_m ：两条曲线附近的切换点。

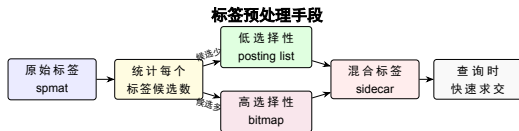
标定方式

- 采样选择性点，测 prefilter / graph 在 recall@10 $\geq$ 98 下的延迟。
- 取两路延迟相近处为 τ_m ，写入索引元数据。
- 当前采用保守界线：前台搜索线程数不超过 14 时启动标定查询；标定期间暂停插入/删除。

16 线程依据

- 图中取 $L = 200$ 下最慢 graph 点：range-u75, recall@10 为 98.60。
- 仅展示 avg、p95 与 10ms budget；1-16 线程均低于预算。

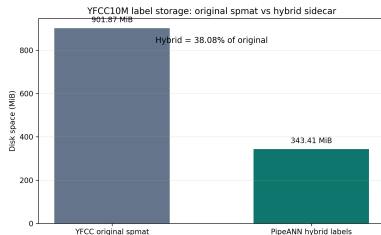




按标签选择性选择表示：小集合用 posting list 省空间，大集合用 bitmap 保持稳定求交速度。

标签表示

- 原始标签采用 spmat 格式：行是向量、列是标签，非零表示持有该标签。
- 预处理后生成混合标签结构：按标签候选规模选择 posting list 或 bitmap。
- posting list: `std::vector<uint32_t>`，保存向量 id。
- bitmap: `std::vector<uint64_t>`，每个 bit 表示一个向量。



实验结果

- YFCC10M 原始 spmat 标签文件为 901.87 MiB。
- 构建后的混合标签 sidecar 为 343.41 MiB，约为原始 spmat 的 38.08%。
- 对比口径只保留 YFCC 原始 spmat 与构建后混合标签。

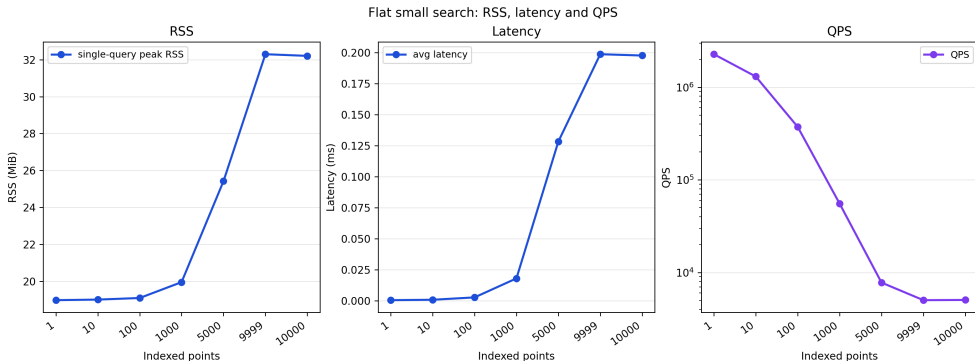
结论：混合标签预处理在保持标签语义的同时，带来明确的磁盘空间收益。

需求 0：从 0 插入与阈值前搜索

从 0 数据量向索引中插入向量时，进入纯内存模式；0–10k 向量直接插入内存数组，并执行精确近邻搜索。该阶段不训练 PQ、不生成磁盘图；索引中向量数超过 10k 后，再一次性落盘为磁盘图索引。

SIFT1M 前 10k 向量实验中，10k 以内全部保持纯内存模式；单查询峰值 RSS 最高约 **32.3 MiB**，接近 30MB 要求；平均搜索延迟最高约 **0.20 ms**，显著低于 10ms 延迟要求，10k 附近 QPS 约 **5k**。

结论：已支持从 0 数据量开始插入；落盘前延迟满足要求，内存占用与 30MB 目标基本一致。

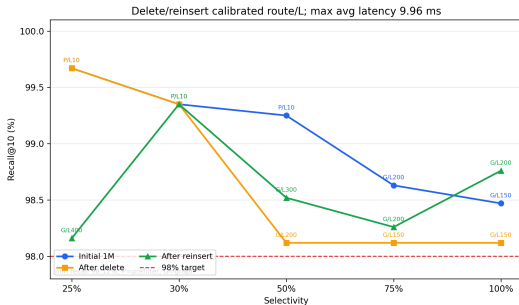
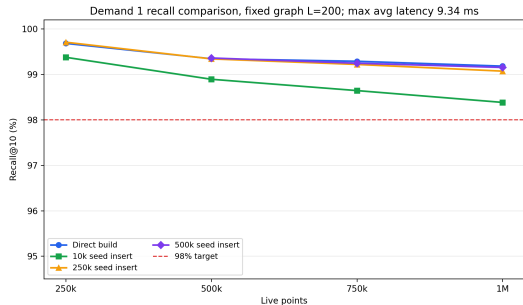


需求 1：召回率对比

左图：直接构建 vs 增量插入。分别直接构建 250k、500k、750k、1M 索引，并与从不同初始规模索引插入到相同规模后的索引比较；两类索引在固定 $L = 200$ 下执行搜索。 $L=200$ 已确认所有 avg latency $< 10\text{ms}$ ，最低 recall@10 为 98.38。

右图：删除 / 重插入后的图质量。先构建 1M 索引，删除 250k 向量得到 750k 索引，再插入 250k 向量回到 1M。per-bucket calibrated route/ L 的高选择性结果全部 ≥ 98 ，最大 avg latency 为 9.96ms。

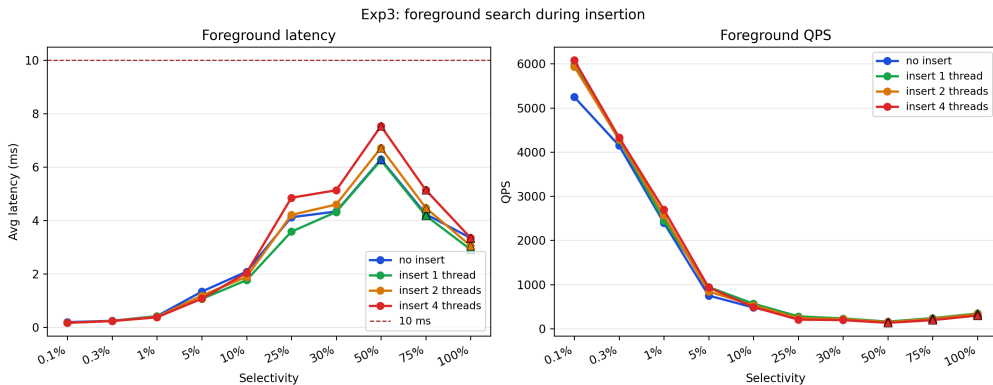
结论：PQ16/R116 在 $L=200$ 或 per-bucket calibration 口径下满足 recall@10 > 98 ，且延迟保持在 10ms 预算内。



需求 2：插入期间前台查询

在初始 1M 索引上插入向量的同时执行前台搜索，测量搜索延迟与 QPS，结果如下图。

结论：1-2 个插入线程下影响较小；4 个插入线程会带来更明显的尾延迟波动。

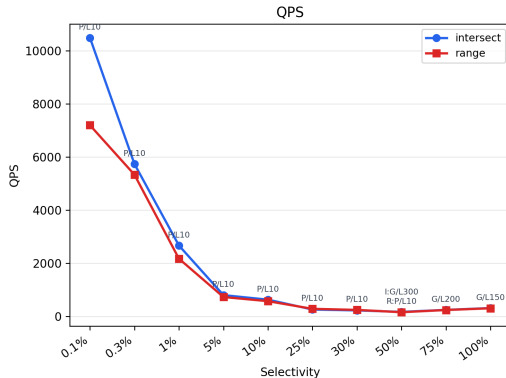
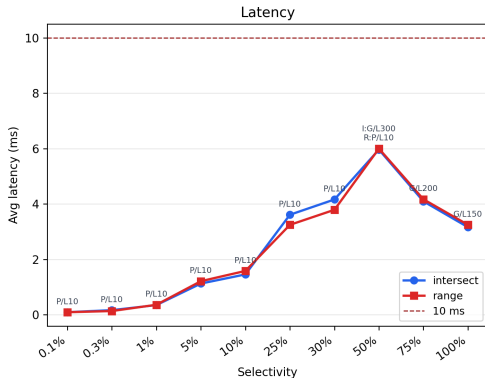


需求 3：选择性查询延迟/QPS

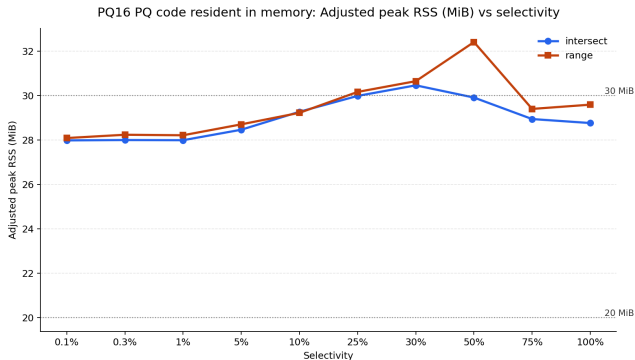
1M 索引；等值 / 范围查询；PQ16 常驻内存；per-bucket 选择满足 $\text{recall@10} \geq 98$ 的最快 route/L。

结论：20 个选择性点全部 $\text{recall@10} \geq 98$ ，最大 avg latency 为 6.00ms。

Exp4: Direct 1M intersect/range search, selected route and minimum L for $\text{recall@10} \geq 98\%$



需求 4: 单查询 adjusted RSS



測量方法

使用 `/usr/bin/time` 记录搜索进程峰值 RSS，并按实际文件大小扣除 query bin 与 GT。

实测结果

PQ16 + PQ code 常驻内存时，20 个点 adjusted RSS 为 **27.98–32.41 MiB**；标签、sidecar、PQ code resident pages 与运行 buffer 均计入。

结论：PQ16 + PQ code 常驻内存满足 30MiB 左右的生产口径。

R=116/PQ16 下，额外索引文件主要来自磁盘图；1M 数据规模下额外空间约为原始向量的 2.03 倍。

规模	原始向量	磁盘图	PQ 压缩向量	PQ 码本	标签位图	额外合计	额外/原始
250k	122.07	244.14	3.81	0.13	0.22	248.31	2.03x
500k	244.14	488.29	7.63	0.13	0.44	496.49	2.03x
750k	366.21	732.43	11.44	0.13	0.67	744.67	2.03x
1M	488.28	976.57	15.26	0.13	0.89	992.85	2.03x

单位：MiB；额外/原始为额外合计与原始向量的比值。

THE END

谢谢

